Bridge to CS: Homework #9

NYU Tandon School of Engineering

Submission Instructions

1. For this assignment, you should turn in 4 `.cpp` files. Name these files: `hw9_q1.cpp`, `hw9_q2.cpp`, etc.
2. You should submit your homework in the Gradescope system.
3. Pay special attention to the style of your code. Indent your code correctly, choose meaningful names for your variables, define constants where needed, choose most suitable control statements, break down your solutions by defining functions, etc.

Question 1

Write a program that will receive a string and output the number of words in the line and the number of occurrences of each letter.

Definitions and Requirements:

- Use this as the function definition.
  string wordAndLetterCount(string input);
- Define a word to be any string of letters that is delimited at each end by either whitespace, a period, a comma, or the beginning or end of the line.
- You can assume that the input consists entirely of letters, whitespace, commas, and periods.
- When outputting the number of letters that occur in a line, be sure to count upper and lowercase versions of a letter as the same letter.
- Output the letters in alphabetical order and list only those letters that do occur in the input line.
- Return the output as one string with new lines characters to separate the words count and character counts.

Example:

```
Input:
I say Hi.

Visual Representation of Output:
3 words
```

```
1 a
1 h
2 i
1 s
1 y

Output as a String:
"3\twords\n1\ta\n1\th\n2\ti\n1\ts\n1\ty"

Make note of the use of tab characters(\t).
```

Notes:

1. Think how to break down your implementation into functions.
2. Pay attention to the running time of your program. If the input line contains n characters, an efficient implementation would run in linear time (that is $\Theta(n)$).

Question 2

Two strings are anagrams if the letters can be rearranged to form each other. For example, "Eleven plus two" is an anagram of "Twelve plus one". Each string contains one 'v', three 'e's, two 'l's, etc.

Write a program that determines if two strings are anagrams. The program should not be case sensitive and should disregard any punctuation or spaces.

Notes:

1. Use the following function definition
   bool areAnagrams(string s1, string s2);
2. Think how to break down your implementation into functions.
3. Pay attention to the running time of your program. If each input string contains n characters, an efficient implementation would run in linear time (that is $\Theta(n)$).

Question 3

In this question, you will write four versions of a function `getPosNums` that gets an array of integers `arr`, and its logical size. When called it creates a new array containing only the positive numbers from `arr`.

For example, if `arr = [3, -1, -3, 0, 6, 4]`, the functions should create an array containing the following 3 elements: `[3, 6, 4]`.

The four versions you should implement differ in the way the output is returned. The prototypes of the functions are:

a) `int* getPosNums1(int* arr, int arrSize, int& outPosArrSize)`
   Returns the base address of the array (containing the positive numbers), and updates the output parameter `outPosArrSize` with the array's logical size.

b) `int* getPosNums2(int* arr, int arrSize, int* outPosArrSizePtr)`
   Returns the base address of the array (containing the positive numbers), and uses the pointer `outPosArrSizePtr` to update the array's logical size.

c) `void getPosNums3(int* arr, int arrSize, int*& outPosArr, int& outPosArrSize)`
   Updates the output parameter `outPosArr` with the base address of the array (containing the positive numbers), and the output parameter `outPosArrSize` with the array's logical size.

d) `void getPosNums4(int* arr, int arrSize, int** outPosArrPtr, int* outPosArrSizeP`
   Uses the pointer `outPosArrPtr` to update the base address of the array (containing the positive numbers), and the pointer `outPosArrSizePtr` to update the array's logical size.

Question 4

Implement the function:

```
void oddsKeepEvensFlip(int arr[], int arrSize)
```

This function gets an array of integers `arr` and its logical size `arrSize`. When called, it should reorder the elements of `arr` so that:

1. All the odd numbers come before all the even numbers.
2. The odd numbers will keep their original relative order.
3. The even numbers will be placed in a reversed order (relative to their original order).

Example: If `arr = [5, 2, 11, 7, 6, 4]`, after calling `oddsKeepEvensFlip(arr, 6)`, `arr` will be: `[5, 11, 7, 4, 6, 2]`.

Implementation requirements:

1. Your function should run in linear time. That is, if there are n items in `arr`, calling `oddsKeepEvensFlip(arr, n)` will run in $\Theta(n)$.